

CryptoTrack Backend Process

This document summarizes the backend architecture, implementation details, and interview preparation points for the CryptoTrack project.

1. Project Overview

The backend is a Node.js Express API that supports user authentication, watchlist and portfolio management, and currency exchange rate proxying. MongoDB is used as the database, while Passport.js handles local and JWT-based authentication.

2. Key Backend Files

- Server/server.js - main API server, route definitions, and CORS configuration.
- Server/db.js - Mongoose database connection setup and lifecycle logging.
- Server/auth.js - Passport local and JWT strategies for login and protected routes.
- Server/models/Users.js - User schema, password hashing, watchlist, and portfolio storage.

3. Environment and Startup

The backend uses dotenv to load environment variables from a .env file. Important variables include:

- MONGODB_URI - MongoDB connection string.
- JWT_SECRET - secret used to sign JSON Web Tokens.
- CLIENT - frontend URL allowed for CORS in production.

Start the backend with: node server.js

4. CORS and Local Development

CORS is configured with a custom origin checker that allows local frontend origins on ports 5173 and 5174 in addition to the configured CLIENT origin. This enables secure cross-origin requests during development.

5. Authentication Flow

Authentication uses Passport to separate login and authorization logic:

- POST /register - creates a new user and hashes the password with bcrypt.
- POST /login - verifies credentials and returns a signed JWT valid for 24 hours.
- Protected routes use passport.authenticate("jwt") to validate the Authorization bearer token.

6. User Model and Data Storage

The User model stores:

- username - unique login identifier.
- password - stored as a bcrypt hash.
- watchlist - array of coin IDs saved by the user.
- portfolio - Map of coin IDs to an object with totalInvestment and coins count.

7. API Endpoints

- GET / - health check endpoint returning API status.
- GET /currency - proxy endpoint to retrieve USD-based exchange rates from Frankfurter API.
- POST /register - user registration with password hashing.

- POST /login - user login, JWT issuance, and response with user details.
- GET /watchlist - returns the authenticated user watchlist.
- GET /portfolio - returns the authenticated user portfolio data.
- PUT /watchlist/add - adds a coin to the authenticated user watchlist.
- PUT /watchlist/remove - removes a coin from the authenticated user watchlist.
- PUT /portfolio/update - updates the authenticated user portfolio for buy/sell operations.

8. Portfolio Update Logic

This endpoint handles portfolio changes while keeping investment and holding values consistent. It supports both buy and sell operations with validation:

- If the coin already exists, update its coin quantity and total investment.
- Selling coins subtracts the amount and adjusts the remaining investment proportionally.
- If holdings go to zero, the coin is removed from the portfolio map.
- Input validation requires totalInvestment and coins values to be numeric.

9. Interview Preparation Questions

- How does the backend authenticate users and protect routes?
- Why is JWT chosen over session-based authentication?
- How is password hashing implemented in the User model?
- What is the purpose of the /currency proxy endpoint?
- Why did you configure custom CORS origin checking?
- How does the portfolio update endpoint handle buy and sell transactions?
- Why use a Map for the portfolio field in MongoDB?
- How does dotenv improve configuration management?